Question 1

Assumptions: One customer can only make one order at a given date and time.
Each order can be completed by only one employee.
Parts of the same type are stored on the same shelf, and thus have the same cage code. A part can be of only one type.

When all the data is shown in one table, the data is not in a Normal Form. There are several attributes that contain multiple values in one row of data. The composite primary key is customerNumber, date and time because these attributes can distinguish one order form from any other. This relies on the assumption that a customer can make at most one order at a given date and time, regardless of what parts they buy and which employee helps them.

order_form											
customerNumber (PK)	customerName	customerType	**date** (PK)	**time** (PK)	employeeName	partNumber	partName	partType	partCageCode	partQuantityOrdered	partUnitPrice
H G54587	Jeff Peterson	Consumer	7/1/2024	10:30am	D. Harrison	10654, 10456, 10776, 10657	Float Control, Modulator, Hose Assembly, Float Assembly	Plumbing, Electrical, Plumbing, Plumbing	G413, H433, G413, G413	4, 3, 7, 5	12, 7, 9, 10

Below, the data is now in **1NF.** There are no longer any repeating groups because the table now has four rows, one for each part bought by this customer in this order, which allows all of the values to be atomic. Also, partNumber has been added to the composite primary key because each row now represents one part, which is part of an order that may or may not contain other parts. In the current data, all parts are part of the same order.

order_component											
customerNumber (PK)	customerName	customerType	**date** (PK)	**time** (PK)	employeeName	**partNumber** (PK)	partName	partType	partCageCode	partQuantityOrdered	partUnitPrice
H G54587	Jeff Peterson	Consumer	7/1/2024	10:30am	D. Harrison	10654	Float Control	Plumbing	G413	4	12
H G54587	Jeff Peterson	Consumer	7/1/2024	10:30am	D. Harrison	10456	Modulator	Electrical	H433	3	7
H G54587	Jeff Peterson	Consumer	7/1/2024	10:30am	D. Harrison	10776	Hose Assembly	Plumbing	G413	7	9
H G54587	Jeff Peterson	Consumer	7/1/2024	10:30am	D. Harrison	10657	Float Assembly	Plumbing	G413	5	10

Below, the data is now in **2NF,** which requires there to be no partial dependencies. Since customerName and customerType only depend on customerNumber, I created a customer table for customerNumber plus those two attributes. Similarly, a part's name, type, cageCode and unitPrice only depend on the part itself (partNumber), so I created a separate part table with those four attributes and the partNumber PK. quantityOrdered depends on all parts of the primary key (it depends on the specific order being made by a customer at a given day and time, as well as which part is being bought), so this attribute is left in a table with the whole primary key. employeeName is put in the order table since it is dependent solely on the three attributes associated with a specific order (customerNumber, date and time). I'm assuming that each order can be completed by only one employee (and thus an employee can only complete one order at a given day and time).

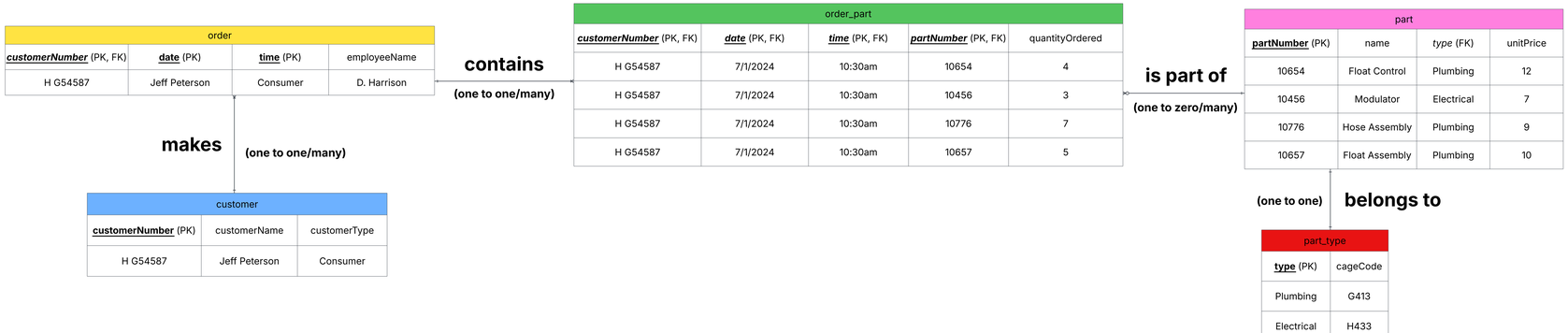
customer		
customerNumber (PK)	customerName	customerType
H G54587	Jeff Peterson	Consumer

order			
customerNumber (PK, FK)	**date** (PK)	**time** (PK)	employeeName
H G54587	Jeff Peterson	Consumer	D. Harrison

part				
partNumber (PK)	name	type	cageCode	unitPrice
10654	Float Control	Plumbing	G413	12
10456	Modulator	Electrical	H433	7
10776	Hose Assembly	Plumbing	G413	9
10657	Float Assembly	Plumbing	G413	10

order_part				
customerNumber (PK, FK)	***date*** (PK, FK)	***time*** (PK, FK)	***partNumber*** (PK, FK)	quantityOrdered
H G54587	7/1/2024	10:30am	10654	4
H G54587	7/1/2024	10:30am	10456	3
H G54587	7/1/2024	10:30am	10776	7
H G54587	7/1/2024	10:30am	10657	5

To get the data into **3NF**, all transitive dependencies must be removed. The only transitive dependency is in the part table: I am assuming that a part's cageCode is dependent on its type, based on the patterns in the data and on existing conventions in real life (all parts of the same type are on the same section, the way inventory is typically organized). I created a new table called part_type, which has type as the PK and cageCode as an attribute. Also, now, in the part table, *type* is an FK, since it references the PK **type** from the new part_type table.



Question 2

Assumptions: A therapist can only work at one branch maximum per day (as stated in question).
Each appointment consists of only one patient and one therapist.
In the 3NF data, there can be one to zero/many relationships because a patient or therapist can exist in the database system without currently having any appointments.

When all the data is shown in one table, the data is not in a Normal Form. The attribute appointmentDateTime contains multiple values in each row of data (**note**: in the question, date and time are separated into two columns but are still in the same box, so I interpreted this as one attribute). The composite primary key is staffNo, patNo and appointmentDateTime because these three attributes together define a unique appointment (who is present and when it is).

appointment					
staffNo (PK)	therapistName	**patNo** (PK)	patName	**appointmentDateTime** (PK)	branchNo
S1011	Fred Smith	P100	Lily White	9/12/2022, 10:00	M15
S1011	Fred Smith	P105	Jill Baker	9/12/2022, 12:00	M15
S1024	Heidi Pierce	P108	Andy McKee	9/12/2022, 10:00	Q10
S1024	Heidi Pierce	P108	Andy McKee	9/14/2022, 14:00	Q10
S1032	Richard Levin	P105	Jill Baker	9/14/2022, 16:30	M15
S1032	Richard Levin	P110	Jimmy Winter	9/15/2022, 18:00	B13

Below, the data is now in **1NF**. I got rid of the repeating group mentioned above by separating the appointmentDateTime attribute into two attributes: appointmentDate and appointmentTime. There is still only one table, but now one more column and all the values are atomic. The composite primary key is now staffNo, patNo, appointmentDate and appointmentTime, but the logic is the same. These four attributes together define a unique appointment.

appointment						
staffNo (PK)	therapistName	**patNo** (PK)	patName	**appointmentDate** (PK)	**appointmentTime** (PK)	branchNo
S1011	Fred Smith	P100	Lily White	9/12/2022	10:00	M15
S1011	Fred Smith	P105	Jill Baker	9/12/2022	12:00	M15
S1024	Heidi Pierce	P108	Andy McKee	9/12/2022	10:00	Q10
S1024	Heidi Pierce	P108	Andy McKee	9/14/2022	14:00	Q10
S1032	Richard Levin	P105	Jill Baker	9/14/2022	16:30	M15
S1032	Richard Levin	P110	Jimmy Winter	9/15/2022	18:00	B13

Below, the data is now in **3NF**. First, I aimed to normalize the data into **2NF** (getting rid of partial dependencies). The attribute therapistName is only dependent on the PK staffNo, so I made a new therapist table. By the same logic, patName is only dependent on the PK patNo, so I created a new patient table. The attribute branchNo is only dependent on staffNo and appointmentDate because a therapist (staff) must stay at one branch on a given day, which means that both the identity of the staff and the date determines which branch an appointment is located at. Thus, I created a new therapist_branch table. These three tables account for all the non-key attributes. Lastly, the appointment table contains staffNo, patNo, appointmentDate and appointmentTime. These are all primary keys as explained earlier, since they distinguish a unique appointment. The attributes staffNo and patNo are also foreign keys because they reference the primary keys of the therapist and patient tables, respectively. And in the therapist_branch table, staffNo is similarly a foreign key because it references the primary key of the therapist table.

This data is now already in **3NF** because there are no transitive dependencies, which occur when a non-key attribute is dependent on another non-key attribute. This is not the case here.

therapist	
staffNo (PK)	therapistName
S1011	Fred Smith
S1024	Heidi Pierce
S1032	Richard Levin

works

(one to zero/many)

appointment			
staffNo (PK, FK)	***patNo*** (PK, FK)	**appointmentDate** (PK)	**appointmentTime** (PK)
S1011	P100	9/12/2022	10:00
S1011	P105	9/12/2022	12:00
S1024	P108	9/12/2022	10:00
S1024	P108	9/14/2022	14:00
S1032	P105	9/14/2022	16:30
S1032	P110	9/15/2022	18:00

attends

(one to zero/many)

patient	
patNo (PK)	patName
P100	Lily White
P105	Jill Baker
P108	Andy McKee
P110	Jimmy Winter

(one to zero/many) **has**

therapist_branch		
staffNo (PK, FK)	**appointmentDate** (PK)	branchNo
S1011	9/12/2022	M15
S1024	9/12/2022	Q10
S1024	9/14/2022	Q10
S1032	9/14/2022	M15
S1032	9/15/2022	B13

Question 3

Assumptions: Each contract only applies to one event, but an event can have one or more contracts (as stated in question).
Every event has at least one contract, since an event being in this database system (for a temp agency) implies that there is at least one contract.
An employee can be in the database system without having any contracts.

When all the data is shown in one table, the data is in **1NF.** All the values are atomic and there are no repeating groups. The composite primary key is eNo and contractNo because these attributes alone distinguish work done by one employee at one event. This is unique because it identifies that one contract applies to one employee.

event_work					
eNo (PK)	**contractNo** (PK)	hours	eName	eventNo	eventLoc
1135	C1024	16	Smith J	H25	Queens
1057	C1024	24	Hocine D	H25	Queens
1068	C1025	28	White T	H4	Yonkers
1135	C1025	15	Smith J	H4	Yonkers
1135	C1026	10	Smith J	H25	Queens

Below, the data is now in **2NF**, which requires eliminating partial dependencies. Since eName is an employee's name, this attribute is dependent solely on eNo (the unique number for an employee), so I made a separate employee table. eventNo (the number identifying an event) and eventLoc (an event's location) are both dependent solely on contractNo, since contractNo represents a single event. This is based on the assumption that each contract only applies to one event, which is stated in the question. Thus, eventNo and eventLoc are dependent on contractNo because they are features of an event, so I made a separate contract table. The attribute hours depends on the entire composite primary key (eNo and contractNo) because it represents how many hours one employee worked at one event. In this employee_contract table, eNo and contractNo are now both also foreign keys because they reference the primary keys of the employee and contract tables, respectively.

employee	
eNo (PK)	eName
1135	Smith J
1057	Hocine D
1068	White T

employee_contract		
eNo (PK, FK)	***contractNo*** (PK, FK)	hours
1135	C1024	16
1057	C1024	24
1068	C1025	28
1135	C1025	15
1135	C1026	10

contract		
contractNo (PK)	eventNo	eventLoc
C1024	H25	Queens
C1025	H4	Yonkers
C1026	H25	Queens

Below, the data is now in **3NF**, which requires eliminating transitive dependencies. There is one transitive dependency: contractNo determines eventLoc only through eventNo. The attribute eventLoc depends on eventNo (importantly, this is independent of contractNo) because the event itself is what determines where that event is located. Since eventNo is the unique identifier for events, eventNo determines eventLoc. Even though one contract can only apply to one event, an event can still have multiple contracts, which means that contractNo is not a reliable identifier for events. I removed the attribute eventLoc from the contract table and created a new event table. The event table contains eventNo as the PK and eventLoc as an attribute.

